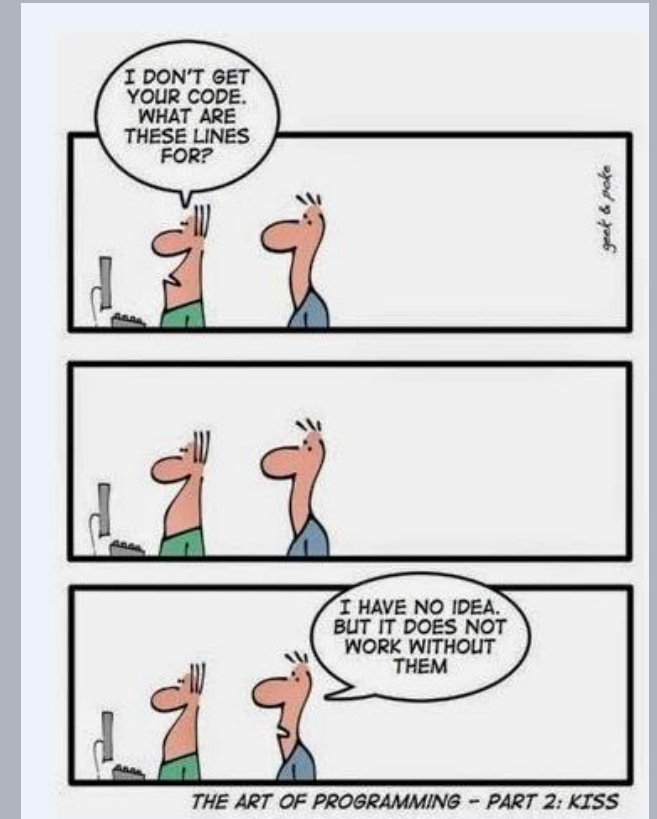


Advanced Software Design Techniques

Mixing C and C++

Prof. Dr.-Ing. Peter Fromm



Content

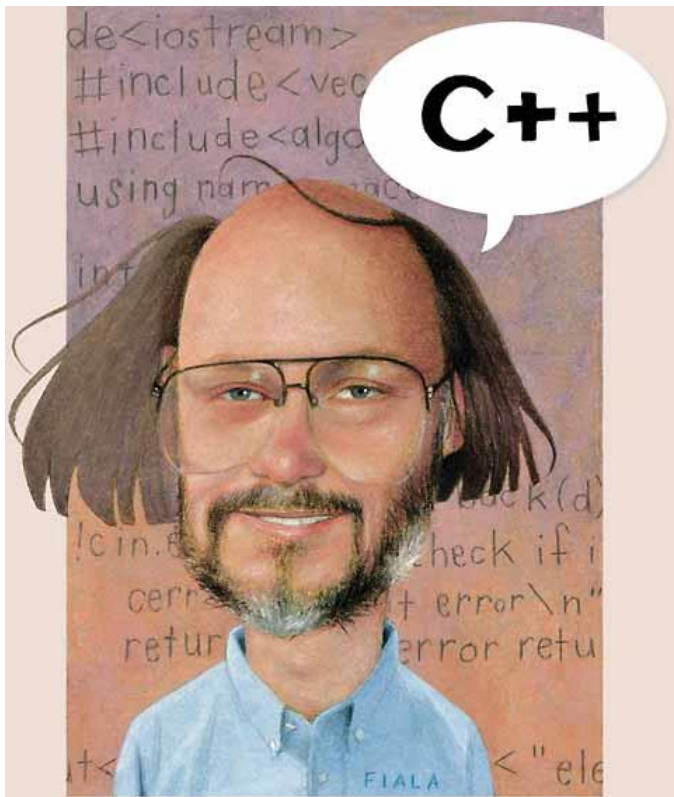
- Motivation and Challenges
- The Linkage Problem
- Magic of POD's
 - Struct and class object initialisation
 - ROMable data
 - Explicit construction
 - C versus C++ casting
- Prepare C code for the C++ compiler
 - Type casting
 - Memory allocation
 - Exceptions
 - String and Math



C and C++

„C makes it easy to shoot yourself in the foot; C++ makes it harder; but when you do it blows your whole leg off.“

Bjarne Stroustrup



Why should we mix C and C++

- Don't mix C++ and C, unless you cannot avoid it.
- C++ provides better encapsulation, stronger type checking and more notational support.
- Many modern libraries come as C++ libraries.
- C++ provides better modelling support.
- Embedded compiler provide more and better C++ support.
- C legacy code especially for embedded systems.
- Many core components like RTOS come as C code.

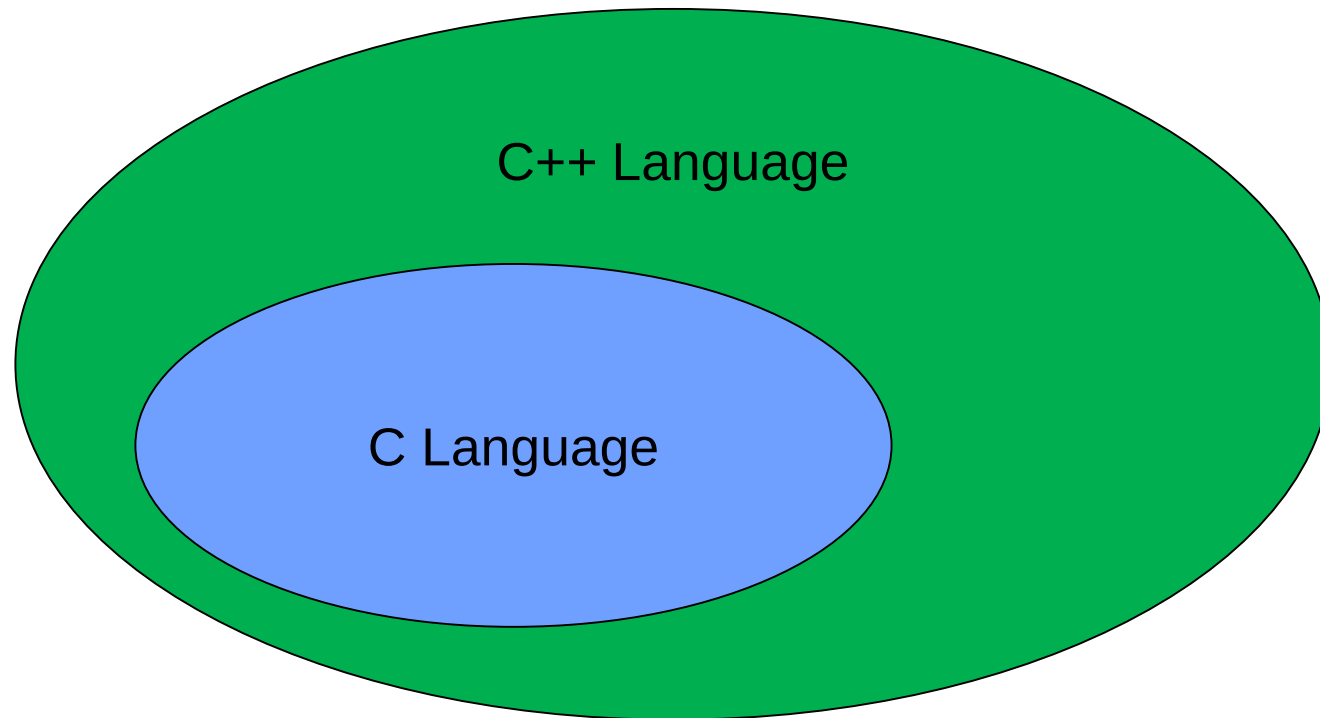


Reasons for C++



Reasons for C

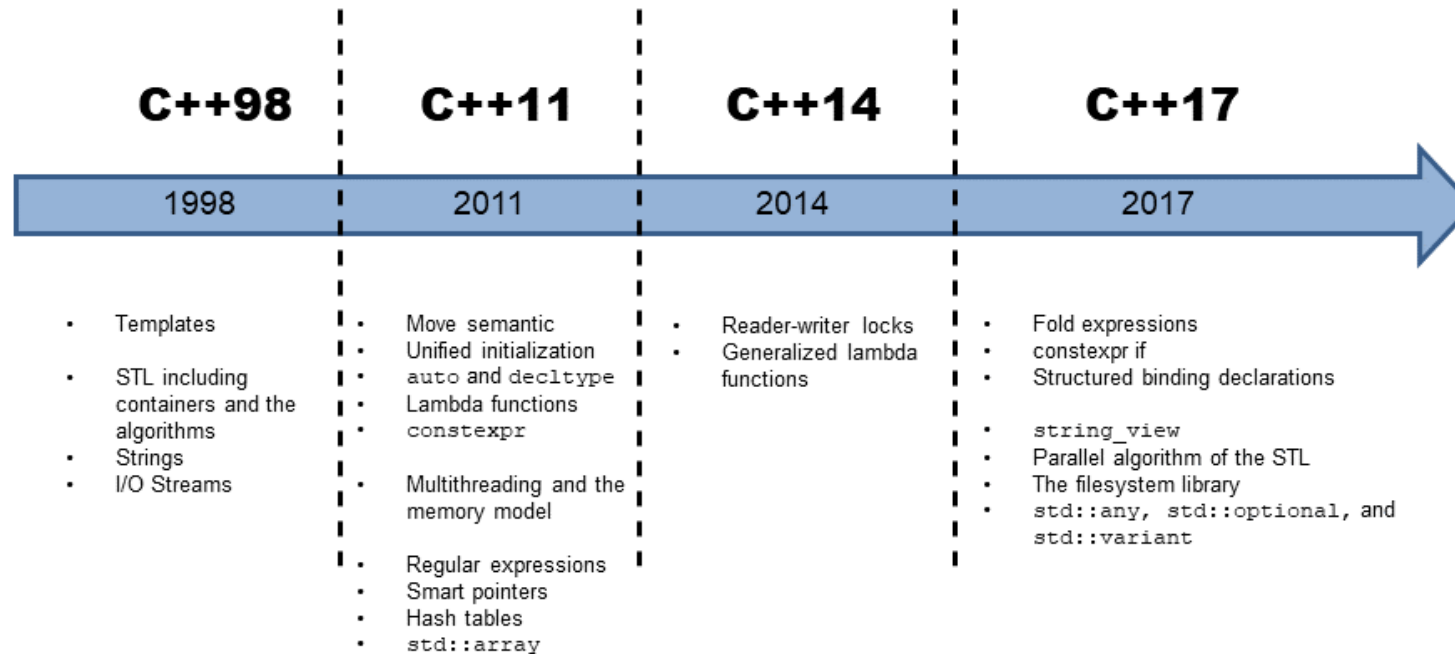
Myth: C++ is an extension of C...



Myth: C++ is an extension of C...

- C++ provides overloaded functions → different linkage conventions (mangled names)
- C++ provides a stricter type checking → C code cannot be compiled under C++
- Structs (and enums) have different conventions under C and C++
- Const is a way less stricter recommendation in C++ compared to C
- C++ compiler comes with complex ideas to cast data types (including usage of the constructor)
- Runtime behaviour e.g. when using C++ exceptions may be undefined in C
- Threadsafety is more complex in C++

Modern History of C++



<https://gcc.gnu.org/projects/cxx-status.html>

Source: <https://www.modernescpp.com/index.php/what-is-modern-c>

To make things even more complex – C++ comes in different flavours

Example Green Hills Embedded C++ Compiler

Feature	Standard	EEC++	EC++
Exception support	Optional	Optional	Optional
Complex numbers	C++	EC++	EC++
Streams and buffers	C++	EC++	EC++
string	C++	EC++	EC++
wchar_t	Yes	Yes	Yes
cin, cout	Yes	Yes	Yes
Template support	Yes	Yes	No
Standard container classes	Yes	Yes	No
Namespace support	Yes	Yes	No
std:: namespace	Yes	Yes	No
bitset<>	Yes	Yes	No
<algorithm>, <functional>, <memory>	Yes	Yes	No
mutable keyword	Yes	Yes	No
static_cast<>, const_cast<>, reinterpret_cast<>	Yes	Yes	No
Multiple inheritance	Yes	No	No
dynamic_cast<>	Yes	No	No
<locale>	Yes	No	No
<limits>	Yes	No	No
wcin, wcout, etc.	Yes	No	No
cerr, clog	Yes	No	No
iostream, stringstream	Yes	No	No
<typeinfo>	Yes	No	No

Mangled names or the linkage problem

C Compiler

```
void foo()  
{  
    cout << "foo()" <<endl;  
}
```

Check the mapfile. Xlinker Option -Map=output.map

```
void foo(int a)  
{  
    cout << "foo(int a)" <<endl;  
}
```

..\myCode\main.cpp:18:6: **error**: conflicting declaration of C function 'void foo(int)'

The linker only creates a single symbol, called “foo”. If we declare another function with the same name, we have a conflict.

256	0x0000000000401410	mainCRTStartup
257	0x0000000000401520	atexit
258 .text	0x0000000000401540	0x20 C:/tool/common/Git/eclipse/m
259	0x0000000000401540	__gcc_register_frame
260	0x0000000000401550	__gcc_deregister_frame
261 .text	0x0000000000401560	0xf0 myCode\main.o
262	0x0000000000401560	foo
263	0x0000000000401591	main
264 .text	0x0000000000401650	0x8 C:/tool/common/Git/eclipse/m
265	0x0000000000401650	std::basic_ostream<char,
266 .text	0x0000000000401658	0x8 C:/tool/common/Git/eclipse/m
267	0x0000000000401658	std::basic_ostream<char,
268 .text	0x0000000000401660	0x0 C:/tool/common/Git/eclipse/m

Mangled names or the linkage problem

C++ Compiler

```
void foo()  
{  
    cout << "foo()" <<endl;  
}
```

```
void foo(int a)  
{  
    cout << "foo(int a)" <<endl;  
}
```

The linker only creates two names “foo()” and “foo(int)”, including the parameter list. This is called a mangled name.

```
258 .text      0x0000000000401540  
259           0x0000000000401540  
260           0x0000000000401550  
261 .text      0x0000000000401560  
262           0x0000000000401560  
263           0x0000000000401591  
264           0x00000000004015c5  
265 .text      0x0000000000401690  
266           0x0000000000401690
```

```
0x20 C:/tool/common/Git/eclipse/mingw/mingw64  
    __gcc_register_frame  
    __gcc_deregister_frame  
0x130 myCode\main.o  
    foo()  
    foo(int)  
    main  
0x8 C:/tool/common/Git/eclipse/mingw/mingw64  
    std::basic_ostream<char, std::char_t
```



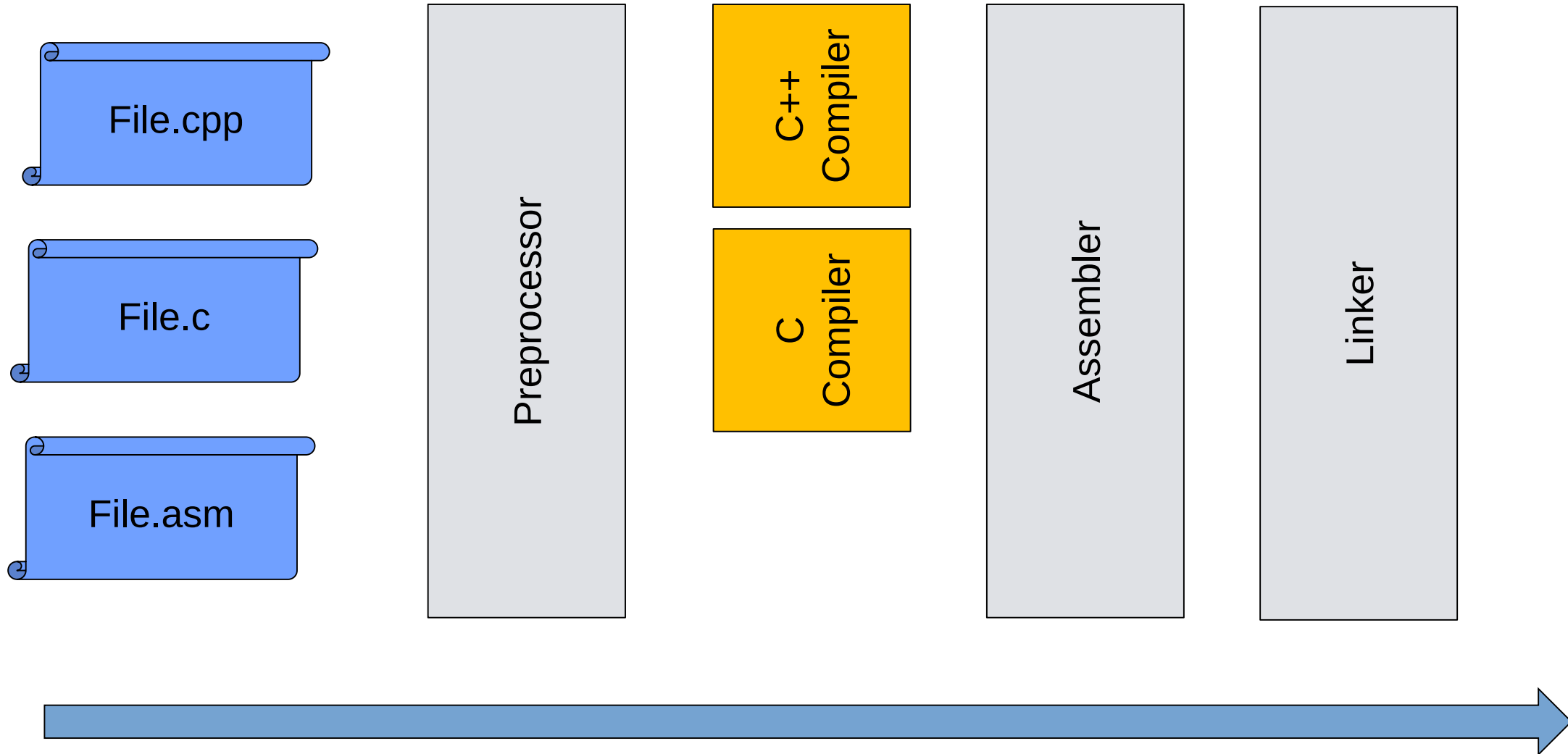
Mangled names or the linkage problem

- Note the different names for foo() when using the C and C++ compiler:
 - foo versus foo()
 - C compiler does not add bracket!
- From the perspective of the linker, we have 3(!) different functions!

256	0x0000000000401410	mainCRTStartup	
257	0x0000000000401520	atexit	
258 .text	0x0000000000401540	0x20 C:/tool/common/Git/eclipse/m	
259	0x0000000000401540	__gcc_register_frame	
260	0x0000000000401550	__gcc_deregister_frame	
261 .text	0x0000000000401560	0xf0 myCode\main.o	
262	0x0000000000401560	foo	
263	0x0000000000401591	258 .text	0x0000000000401540
264 .text	0x0000000000401650	259	0x0000000000401540
265	0x0000000000401650	260	0x0000000000401550
266 .text	0x0000000000401658	261 .text	0x0000000000401560
267	0x0000000000401658	262	0x0000000000401560
268 .text	0x0000000000401660	263	0x0000000000401591
		264	0x00000000004015c5
		265 .text	0x0000000000401690
		266	0x0000000000401690

0x20 C:/tool/common/Git/eclipse/mingw/mingw64	
__gcc_register_frame	
__gcc_deregister_frame	
0x130 myCode\main.o	
foo()	
foo(int)	
main	
0x8 C:/tool/common/Git/eclipse/mingw/mingw64	
std::basic_ostream<char, std::char_t	

Build steps - simplified



Calling C functions from C and C++ from a C++ file

C file

```
main()  
{  
    foo();  
}
```

Searches for
“foo”

C file

```
foo()  
{  
    ...  
}
```

Provides “foo”

C++ file

```
main()  
{  
    foo();  
}
```

Searches for
“foo()”

C file

```
foo()  
{  
    ...  
}
```

Provides “foo()”

Calling C functions from a C++ file

C++ file

```
main()  
{  
    foo();  
}
```

Searches for
"foo()"

C file

```
foo()  
{  
    ...  
}
```

Provides "foo"

Linker error, not compiler
error!

C:/tool/common/Git/eclipse/mingw/mingw64/bin/../../lib/gcc/x86_64-w64-mingw32/9.2.0/../../../../x86_64-w64-mingw32/bin/ld.exe: myCode\main.o: in function `main':

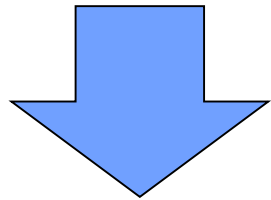
D:\50_HDA\60_Lehre\40_Vorlesungen\25_ADT\20_Lecture_work\120_ExternC\Debug/../../myCode/main.cpp:18: undefined reference to `foo()'
collect2.exe: error: ld returned 1 exit status

Calling C functions from a C++ file

C++ file

```
main()  
{  
    foo();  
}
```

Searches for
"foo()"



Searches for
"foo"

C file

```
foo()  
{  
    ...  
}
```

Provides "foo"

This cannot be
changed. C
compiler can only
provide C-
Linkage!

```
//In cfile.h...  
//This will tell the file including  
this header that foo is linked with C-  
linkage!  
extern "C"  
{  
  
void foo();  
  
}
```

Calling C functions from a C++ file

- Problem: extern "C" is only known to a C++ compiler, i.e. this command must be hidden in case the header file is included from a C-file

```
#ifndef CFILE_H_  
#define CFILE_H_
```

```
#ifdef __cplusplus
```

```
    extern "C"  
    {
```

```
#endif
```

```
        void foo();
```

```
#ifdef __cplusplus  
    }
```

```
#endif //extern "C"
```

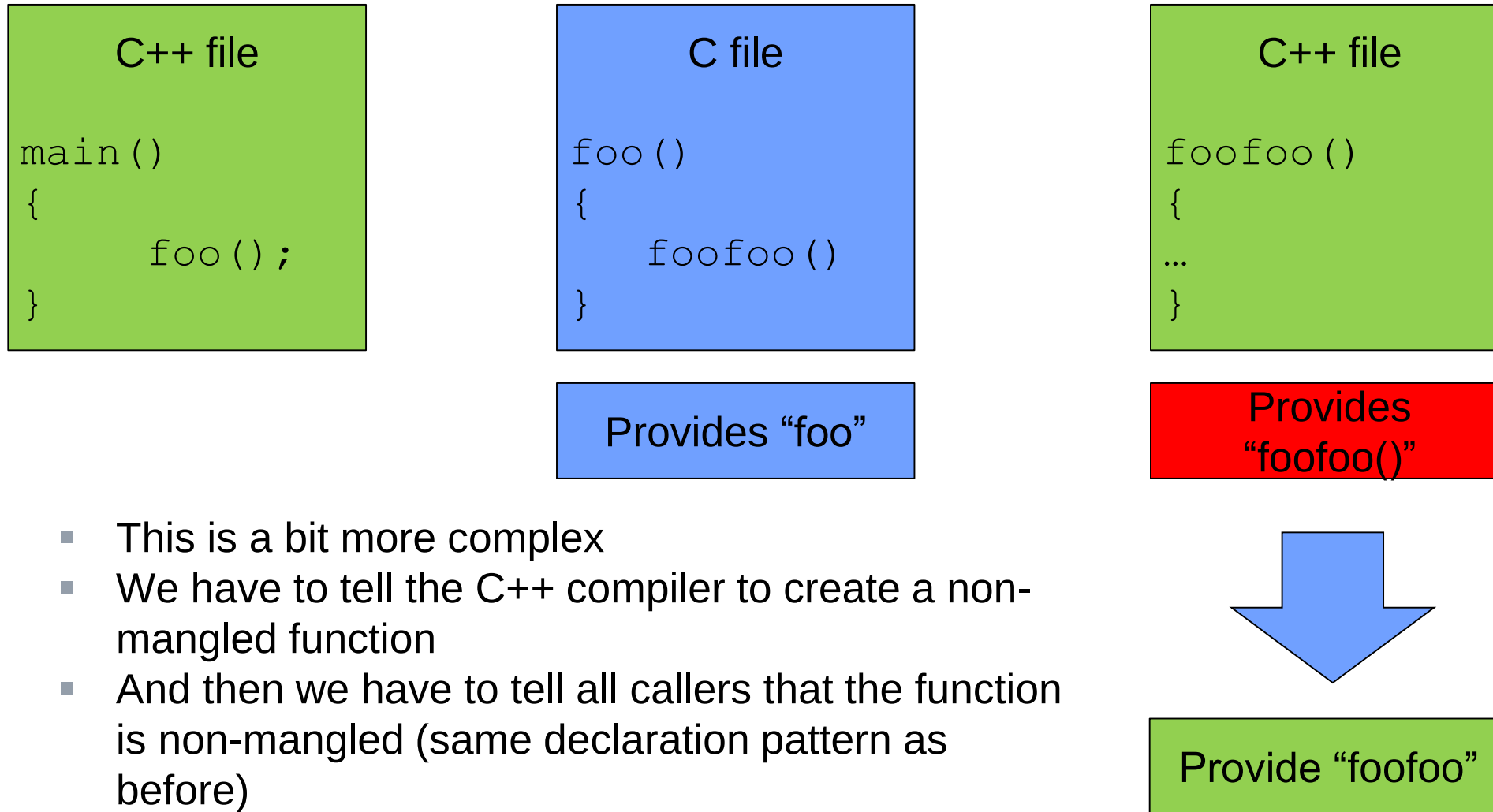
```
#endif /* CFILE_H_ */
```

Only for C++ includes

Declare the function with C-linkage

End of C-linkage section

Calling C++ functions from a C file

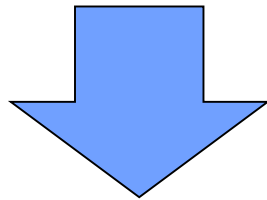


Calling C++ functions from a C file

C++ file

```
foofoo()  
{  
...  
}
```

Provides
"foofoo()"



Provide "foofoo"

```
//In cppfile.cpp  
extern "C"  
{  
  
    void foofoo()  
    {  
        cout << "cppfile foofoo with C-Linkage" << endl;  
    }  
  
}
```

Calling C++ class method from a C file

- From a linkage perspective, more or less the same challenge
 - Unfortunately methods cannot be compiled having C-linkage
 - A C-Compiler does not know how to call a method

```
extern "C"  
{  
  
void CfoofooWrapper(char* text)  
{  
    Cfoofoo obj(text);  
    obj.print();  
}  
}
```

We need an additional wrapper function in the cpp file, having external linkage, which will create an object and call the method.

Plain old Data (POD)

Handling data in C and C++ differs

Data in C

- Relatively simple, linear data structures
- Initialisation by explicit assignment or memcpy (e.g. globals)

Data in C++

- Class becomes a central concept
- Additional elements like constructor, copy constructor, destructor, assignment operator ,...
- A struct is a public class

To support migration from C to C++, POD's are treated similar in C and C++.
A POD is a struct/class only containing “simple” attributes and do not use OO specific features like constructors and inheritance.

POD Initialisation 1

```
typedef struct pod1{
int m_a;
int m_b;

void print()
{
cout << "my pod1_t: " << m_a << ", " << m_b
    << endl;
}
} pod1_t;

pod1_t data1 = {2,3};
```

➔ Value 2,3 (Initialiser List)

```
typedef struct pod1{
int m_a;
int m_b;

pod1()
{
m_a = 99;
m_b = 77;
}

void print()
{
cout << "my pod1_t: " << m_a << ", " << m_b
    << endl;
}
} pod1_t;

pod1_t data1;
```

➔ Value 99, 77 (Constructor)

POD Initialisation 2

```
typedef struct pod1{  
    int m_a;  
    int m_b;
```

```
pod1 ()  
{  
    m_a = 99;  
    m_b = 77;  
}
```

```
void print()  
{  
    cout << "my pod1_t: " << m_a << ", " << m_b  
        << endl;  
}  
} pod1_t;
```

```
pod1_t data1 = {2,3};
```

➔ Will not compile

```
typedef struct pod1{  
    int m_a = 0;  
    int m_b = 0;
```

```
void print()  
{  
    cout << "my pod1_t: " << m_a << ", " << m_b  
        << endl;  
}  
} pod1_t;
```

```
pod1_t data1 = {2,3};
```

➔ Will also not compile

It is initialiser list XOR constructor. Both together cause a conflict.

ROMable data

Often, we want to place global data in a readonly memory section like flash

- For safety reasons (no wild pointer will be able to corrupt it)
- Or because we have more flash compared to RAM
- Or because Flash is cheaper as compared to RAM

```
const char VERSIONSTRING[] = "Version 1.3.6"
```

The Linker will place the string "Version 1.3.6" somewhere in Flash and store the address in VERSIONSTRING.

In C++, this may work with POD's, but const in C++ as compared to C is more of a recommendation for the compiler. It will definitely not work in case a constructor is invoked.

Explicit construction

```
struct s1{
    int m_a;
    int m_b;

    explicit s1(int a=0, int b=0)
    {
        m_a = a;
        m_b = b;
    }

    void print()
    {
        cout << "my pod1_t: " << m_a << ", " << m_b << endl;
    }

} ;

int main (void)
{
    s1 data;
    data = 2; //Why does this work????
    data.print();

}
```

Use explicit constructors to avoid unexpected construction of data!

C style casting versus C++ style casting

```
double gd = 123456789.999;
```

```
// Main program  
int main (void)  
{
```

```
float* pf;  
pf = (float*)&gd; //This will compile and create a completely wrong result, as both data types have  
different sizes and layouts
```

```
pf = static_cast<float*>(&gd); //This will not compile
```

```
cout << "FLoat value: " << *pf << endl;
```

```
return 0;  
}
```

C style casting versus C++ style casting

- The functionality of C style explicit conversion operators is enough for most needs with fundamental data types.
- However, these operators can be applied indiscriminately on classes and pointers to classes, which can lead to code that while being syntactically correct can cause runtime errors.
- C++ style casts perform a stricter type checking, avoiding side effects from undefined casts, resulting in easier detectable compiler errors compared to runtime errors.
 - `static_cast<>`
 - `reinterpret_cast<>`
 - `const_cast<>`

Prefer new C++ style casts over C style casts.

Migration from C to C++

- CPL.1: Prefer C++ to C
- CPL.2: If you must use C, use the common subset of C and C++, and compile the C code as C++
- CPL.3: If you must use C for interfaces, use C++ in the calling code using such interfaces

In order to avoid linkage problems and to make use of the stronger language constructs of C++, a simple migration strategy is to rename all C-files to C++ files.

This code compiles under C

```
#include <stdio.h>

int main(){

    double sq2 = sqrt(2);                // Declaration for sqrt missing

    printf("\nsizeof('\\a\\'): %d\\n\\n", sizeof('a')); // Incorrect use of sizeof

    char c;
    void* pv = &c;
    int* pi = pv;                        // Invalid pointer conversion

    int class = 5;                       // A variable called class

}
```

Especially the stricter type checking of void pointers will require some serious refactoring in many embedded systems!

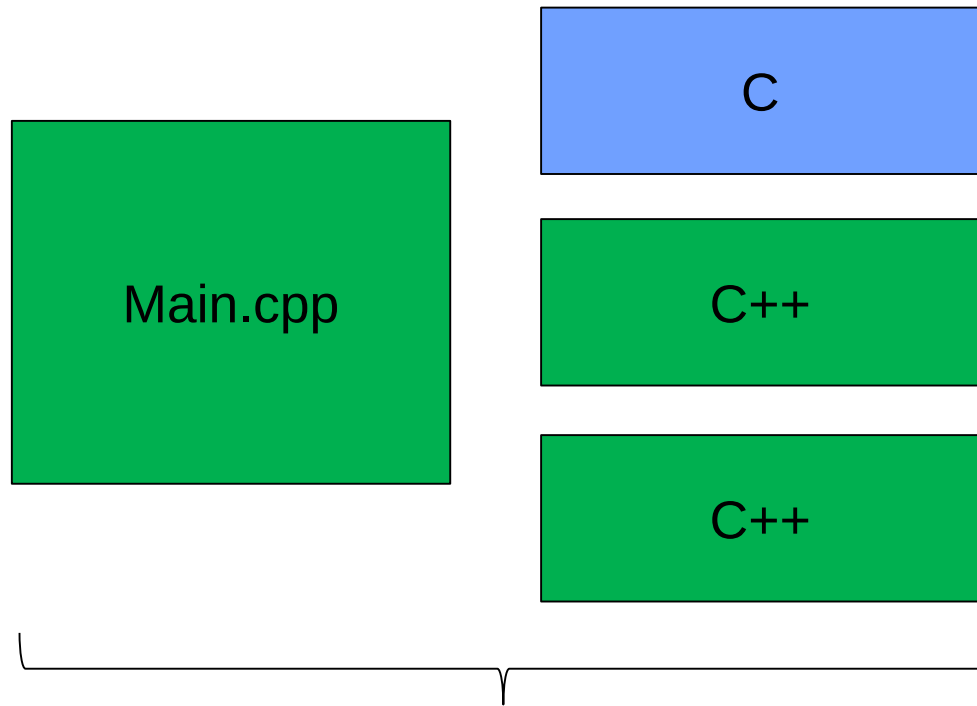
Some more refactoring areas...

- Memory allocation → prefer new and delete over malloc and free
- Exception handling → make sure that exceptions are kept away from C code - catch(...)
- Structs become classes – at some point during the refactoring. Which will cause problems for the system initialisation – no more initialiser lists
- New runtime errors will pop up, as alignment and padding of data may be different
- String data type comes in different flavours
- C and C++ math libs may differ
- ...

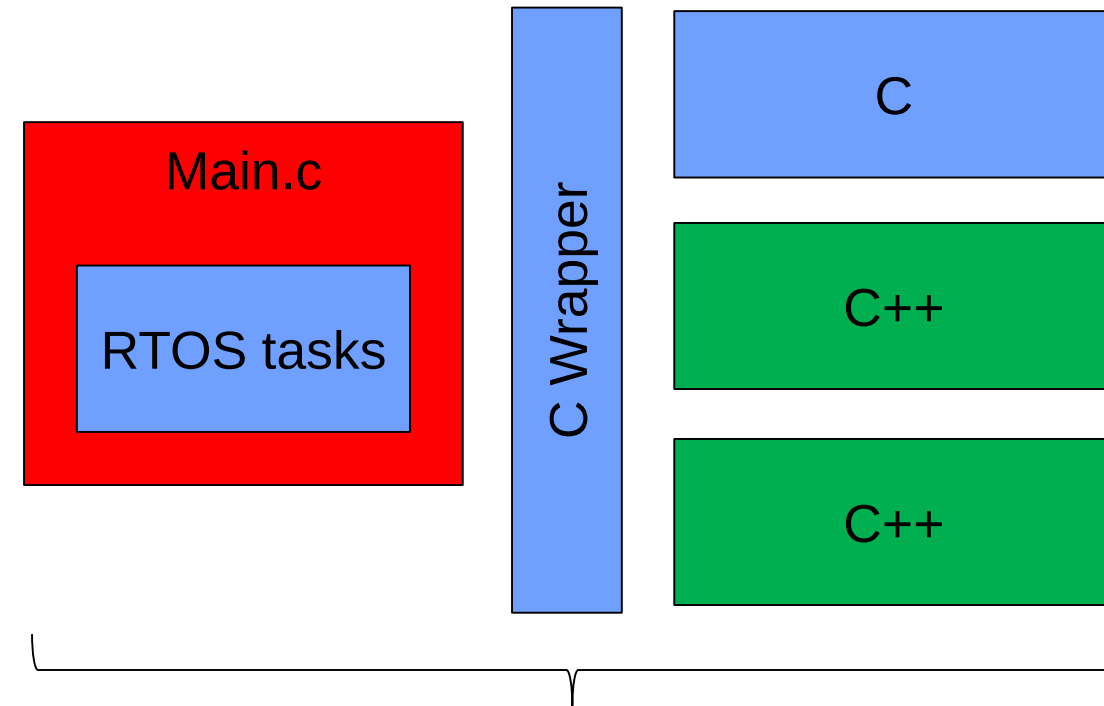
This is not a weekend job!!! And very likely ends up in a domino race.



Master and Servant



Preferred, as we ONLY have to consider C linkage, no wrappers required



In reality, the RTOS unfortunately comes as C source.

Wrapup

- Mixing C and C++ should be avoided if possible
- C++ play a stronger role in embedded systems
- Better modelling and design support, more and more libraries come as C++
- Key components like RTOS will remain C-based for the next couple of years

Mixing C and C++ should be avoided if possible – forget it!
Like every refactoring job, it must be planed. It's not only renaming filenames!